



## INFORME DE GUÍA PRÁCTICA

### **I.PORTADA**

|                                    |                               |
|------------------------------------|-------------------------------|
| Tema:                              | Árboles                       |
| Unidad de Organización Curricular: | BÁSICA                        |
| Nivel y Paralelo:                  | 3 “A”                         |
| Alumnos participantes:             | Edwin Patricio Caraguay Pucha |

|             |                              |
|-------------|------------------------------|
| Asignatura: | .....<br>Estructura de Datos |
| Docente:    | Ing. Mg. José Caiza          |

### **II.INFORME DE GUÍA PRÁCTICA**

#### **2.1 Objetivos**

##### **General:**

Desarrollar habilidades en el trabajo con árboles

##### **Específicos:**

- Desarrollar los ejercicios propuestos en la guía práctica.
- Aplicar los conocimientos adquiridos para definir estructuras de árbol y operaciones.
- Desarrollar operaciones básicas sobre el árbol: inserción de nodos, conteo total de nodos y conteo de hojas.

#### **2.2 Instrucciones**

1. Lleve a cabo las actividades indicadas.
2. Suba a plataforma un informe con el resultado obtenido

#### **2.3 Listado de equipos, materiales y recursos**

Listado de equipos y materiales generales empleados en la guía práctica:

- **Inteligencia Artificial**
- **TAC**
- **Material Bibliográfico**
- **Internet**



TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☒ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial

Otros (Especifique): \_\_\_\_\_

## 2.4 Actividades por desarrollar

Desarrollar lo indicado en archivo adjunto. Luego, elaborar un informe del trabajo llevado a cabo donde resalte los aspectos fundamentales tenidos en cuenta en su propuesta de solución.

## 2.5 Resultados obtenidos

Para esta guía práctica se plantearon 5 ejercicios de código sobre árboles, cada uno diseñado tanto para Java como para C++, se tratará de explicar lo realizado en cada ejercicio y dar pruebas de su ejecución

También para mejor comprensión o visualización de lo realizado en esta guía práctica se deja el link del repositorio en github: <https://github.com/Pato77232/APE3-Estructuras-de-Datos>

### Ejercicio 1

Se necesita implementar una función de contarNodos(), entonces primeramente definimos un condicional para no contar nada si no tenemos nodos que contar, y seguidamente añadimos las validaciones correspondientes para contar el nodo actual y avanzar recursivamente hacia los nodos hijos

**-Implementación y ejecución en Java:**

```
public class Ejercicio1_Basico {
    public static int contarNodos(NodoN raiz) {
        // Si el nodo es null, no contamos nada
        if (raiz == null) {
            return 0;
        } // Contamos el nodo actual y luego contamos recursivamente los nodos de sus hijos
        int count = 1; // Contamos el nodo actual
        for (NodoN hijo : raiz.hijos) {
            count += contarNodos(hijo);
        }

        return count;
    }
}
```

--- Prueba Ejercicio 1 ---

Nodos esperados: 6

Nodos calculados: 6



### -Implementación y ejecución en C++

```
int contarNodos(NodoN* raiz) {
    if (raiz == nullptr) {
        return 0; // Si el nodo es null, no contamos nada
    }
    int count = 1; // Contamos el nodo actual
    for (NodoN* hijo : raiz->hijos) {
        count += contarNodos(hijo); // Contamos los nodos de los hijos recursivamente
    }
    return count;
}
return 0;
```

```
pat077232@redora: ~/Escritorio/
--- Prueba Ejercicio 1 ---
Nodos esperados: 6
Nodos calculados: 6
```

### Ejercicio 2

Se necesita implementar una función de insertar nodos en un árbol binario, igualmente establecemos condicional en caso de que el árbol este vacío, y seguidamente iniciamos compara-

dores de mayor y menor para añadir el nuevo nodo si es menor a la izquierda, es mayor a la derecha hasta que lleguemos a un nodo que ya no tenga hijos.

### -Implementación y ejecución en Java:

```
public static Nodo insertar(Nodo raiz, int valor) {
    // Si el árbol está vacío, creamos un nuevo nodo
    if (raiz == null) {
        return new Nodo(valor);
    }
    // Si el valor a insertar es menor que el valor del nodo raiz, lo insertamos en el subárbol izquierdo
    if (valor < raiz.valor) {
        raiz.izquierdo = insertar(raiz.izquierdo, valor);
    } else if (valor > raiz.valor) { // Si el valor a insertar es mayor que el valor del nodo
        raiz.derecho = insertar(raiz.derecho, valor);
    }
    return raiz;
}
```

```
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
```

### Implementación y ejecución en C++:

```
Nodo* insertar(Nodo* raiz, int valor) {
    if (raiz == nullptr) {
        return new Nodo(valor); // Si el nodo es null, creamos uno nuevo
    }
    if (valor < raiz->valor) {
        raiz->izquierdo = insertar(raiz->izquierdo, valor); // Insertamos en el subárbol izquierdo
    } else if (valor > raiz->valor) {
        raiz->derecho = insertar(raiz->derecho, valor); // Insertamos en el subárbol derecho
    }
    return raiz;
}
```

```
pat077232@redora: ~/Escritorio/ Estructura-de-Datos-Sw
```

### Ejercicio 3

```
--- Prueba Ejercicio 2 ---
Raiz (Esperado 10): 10
Hijo Izquierdo (Esperado 5): 5
Hijo Derecho (Esperado 15): 15
Hijo Izq del 5 (Esperado 3): 3
```

Se necesita una función para calcular la altura del árbol binario, entonces añadimos nuestro condicional

en caso de un árbol vacío y calculamos la altura recorriendo el árbol por izquierda y derecha.

### -Implementación y ejecución en Java:



**UNIVERSIDAD TÉCNICA DE AMBATO**  
**FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL**  
**CARRERA DE SOFTWARE**  
**CICLO ACADÉMICO: ENERO – JUNIO 2026**



```
public static int calcularAltura(Nodo raiz) {  
    if (raiz == null) {  
        return 0; // La altura de un árbol vacío es 0  
    }  
    int alturaIzquierda = calcularAltura(raiz.izquierdo);  
    int alturaDerecha = calcularAltura(raiz.derecho);  
    return 1 + Math.max(alturaIzquierda, alturaDerecha); // Altura del nodo actual + altura máx:  
}
```

--- Prueba Ejercicio 3 ---

Altura esperada: 3

Implementación y ejecución  
en C++:

```
int calcularAltura(Nodo* raiz) {  
    if (raiz == nullptr) {  
        return 0; // La altura de un árbol vacío es 0  
    }  
    int alturaIzquierda = calcularAltura(raiz->izquierdo);  
    int alturaDerecha = calcularAltura(raiz->derecho);  
    return 1 + max(alturaIzquierda, alturaDerecha); // Altura del nodo actual + altura máxima  
}
```

--- Prueba Ejercicio 3 ---

Altura esperada: 3

Altura calculada: 3

Altura de árbol nulo (esperado 0): 0



#### Ejercicio 4

Se necesita implementar una función para recorrer un árbol binario en el recorrido inorden, primeramente establecemos el condicional cuando tenemos un árbol vacío, luego añadimos el funcionamiento del recorrido inorden, primero contamos el nodo izquierdo, luego visita la raíz y finalmente el nodo derecho.

**-Implementación y ejecución en Java:**

```
public class RecorridoInOrder {  
    public static void inOrderAux(Nodo nodo, List<Integer> resultado) {  
        if (nodo == null) {  
            return; // Si el nodo es null, no hacemos nada  
        }  
        // Primero recorremos el subárbol izquierdo  
        inOrderAux(nodo.izquierdo, resultado);  
        // Luego agregamos el valor del nodo actual a la lista de resultados  
        resultado.add(nodo.valor);  
        // Finalmente recorremos el subárbol derecho  
        inOrderAux(nodo.derecho, resultado);  
    }  
  
    public static List<Integer> recorridoInOrder(Nodo raiz) {  
        List<Integer> resultado = new ArrayList<>();  
        inOrderAux(raiz, resultado);  
        return resultado;  
    }  
}
```

--- Prueba Ejercicio 4 ---

Resultado esperado: [1, 2, 3, 4, 5, 6, 7]

```
void inOrderAux(Nodo* nodo, vector<int>& resultado) {  
    if (nodo == nullptr) {  
        return; // Si el nodo es null, no hacemos nada  
    }  
    // Primero recorremos el subárbol izquierdo  
    inOrderAux(nodo->izquierdo, resultado);  
    // Luego agregamos el valor del nodo actual a la lista de resultados  
    resultado.push_back(nodo->valor);  
    // Finalmente recorremos el subárbol derecho  
    inOrderAux(nodo->derecho, resultado);  
}  
  
vector<int> recorridoInOrder(Nodo* raiz) {  
    vector<int> resultado;  
    inOrderAux(raiz, resultado);  
    return resultado;  
}
```

--- Prueba Ejercicio 4 ---

Resultado esperado: [1, 2, 3, 4, 5, 6, 7]

Tu resultado: [1, 2, 3, 4, 5, 6, 7]

dos, de igual manera añadimos un condicional para el caso de un árbol vacío, y para la función se intercambian los nodos izquierdo y derecho

**-Implementación y ejecución en Java**

tación y ejecución en C++

#### Ejercicio 5

Ahora se pide una transformación, concretamente invertir los nodos,



```
public class Ejercicio5_Transformacion {  
    public static Nodo invertir(Nodo raiz) {  
        if (raiz == null) {  
            return null; // Si el nodo es null, no hay nada que invertir  
        }  
        // Intercambiamos los hijos izquierdo y derecho  
        Nodo temp = raiz.izquierdo;  
        raiz.izquierdo = raiz.derecho;  
        raiz.derecho = temp;  
        return raiz;  
    }  
}
```

--- Prueba Ejercicio 5 ---

Antes de invertir:

```
Nodo* invertir(Nodo* raiz) {  
    if (raiz == nullptr) {  
        return nullptr; // Si el nodo es null, no hay nada que invertir  
    }  
    // Intercambiamos los hijos izquierdo y derecho  
    Nodo* temp = raiz->izquierdo;  
    raiz->izquierdo = raiz->derecho;  
    raiz->derecho = temp; // Ahora el hijo izquierdo es el derecho original y viceversa  
    return raiz;  
}
```

mentación  
y ejecución  
en C++

--- Prueba Ejercicio 5 ---

Antes de invertir:

Hijo Izq: 2 | Hijo Der: 3

Después de invertir (Esperado: Izq 3 | Der 2):

Hijo Izq: 3 | Hijo Der: 2

## 2.6 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☒ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

## 2.7 Conclusiones

- El desarrollo de los ejercicios permitió comprender el funcionamiento de los árboles binarios y la importancia de su estructura dentro de las estructuras de datos.
- La implementación de operaciones como inserción de nodos, conteo de nodos, recorrido inorden y cálculo de altura ayudó a reforzar el uso de la recursividad en Java y C++.





- Se comprobó que los árboles binarios facilitan la organización y manipulación eficiente de información, especialmente en procesos de búsqueda y recorrido.

## **2.8 Recomendaciones**

- Practicar constantemente ejercicios de árboles binarios para mejorar la comprensión de recorridos y operaciones recursivas.
- Realizar pruebas con diferentes cantidades de nodos para observar el comportamiento y eficiencia de las operaciones del árbol.
- Mantener una buena organización del código utilizando funciones separadas para cada operación del árbol.

## **2.9 Referencias bibliográficas**

- [1] V. Fritelli, A. Guzmán, y J. Tymoschuk, Algoritmos y estructuras de datos, 2.<sup>a</sup> ed. Córdoba, Argentina: Jorge Sarmiento Editor – Universitas, 2020, 497 pp. [En línea]. Disponible en: enlace activo al URL institucional.
- [2] S. Guardati, Estructuras de datos básicas: programación orientada a objetos con Java, 1.<sup>a</sup> ed. Ciudad de México, México: Alfaomega, 2016, 398 pp. [Código: BFISEI1698a / BFISEI2046a].
- [3] P. A. Sznajdleder, Programación orientada a objetos y estructura de datos a fondo, 1.<sup>a</sup> ed. Buenos Aires, Argentina: Alfaomega, 315 pp. [Código: BFISEI2999a].
- [4] A. Pérez Castaño, Aprender, 1.<sup>a</sup> ed. Barcelona, España: Marcombo, 2018, 213 pp.